

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Описание входных данных.....	6
1.2 Описание выходных данных.....	6
2 МЕТОД РЕШЕНИЯ.....	7
3 ОПИСАНИЕ АЛГОРИТМОВ.....	9
3.1 Алгоритм конструктора класса Device.....	9
3.2 Алгоритм метода show класса Device.....	10
3.3 Алгоритм функции main.....	10
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	13
5 КОД ПРОГРАММЫ.....	18
5.1 Файл Device.cpp.....	18
5.2 Файл Device.h.....	18
5.3 Файл main.cpp.....	19
6 ТЕСТИРОВАНИЕ.....	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	22

1 ПОСТАНОВКА ЗАДАЧИ

В системе центра обработки данных выполняется регистрация устройств. Каждое устройство должно удовлетворять требованиям:

- имя устройства не должно быть пустым;
- мощность устройства должна быть больше нуля;
- уровень нагрузки не должен превышать 100.

При нарушении этих условий система должна генерировать исключения.

Класс Устройство

Поля:

- строковое поле имени устройства;
- целочисленное поле мощности;
- целочисленное поле нагрузки.

Методы:

- конструктор, устанавливающий значения полей, проверяющий корректность данных, при ошибке генерирующий исключение:
 - Если имя пустое, сгенерировать исключение типа `string` со значением "Error: empty name";
 - Если мощность ≤ 0 , сгенерировать исключение типа `int` со значением "-1";
 - Если нагрузка > 100 , сгенерировать исключение типа `double` со значением "100.0"
- метод вывода информации об устройстве в формате "Device::show - > <name> <power> <load>";

Алгоритм работы программы

1. Ввести количество устройств n .
2. Для каждого устройства:

2.1. Ввести значения имени, мощности и нагрузки.

2.2. Попытаться создать объект в блоке try.

2.2.1. Если объект создан успешно, вывести его данные.

2.2.2. Если возникло исключение, обработать его соответствующим блоком catch и вывести сообщение об ошибке.

Формат обработки исключений

- для `string`: `Exception(string): <текст>`
- для `int`: `Exception(int): <значение>`
- для `double`: `Exception(double): <значение>`

Для корректного чтения имени допустимо использовать `getline()`.

1.1 Описание входных данных

Пример:

```
3
Router 50 30
0 20
Server -10 40
```

1.2 Описание выходных данных

Пример:

```
Device::show -> Router 50 30
Exception(string): Error: empty name
Exception(int): -1
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект device класса Device предназначен для хранения информации об устройстве и вывода этих данных;
- функция main для организации работы программы;
- конструкция try-catch для обработки исключений;
- оператор throw для вызова исключений.

Класс Device:

- свойства/поля:
 - поле хранение имени устройства:
 - наименование — name;
 - тип — строковый;
 - модификатор доступа — private;
 - поле хранение мощности устройства:
 - наименование — power;
 - тип — целочисленный;
 - модификатор доступа — private;
 - поле хранение текущей нагрузки устройства:
 - наименование — load;
 - тип — целочисленный;
 - модификатор доступа — private;
- функционал:
 - метод Device — параметризованный конструктор, устанавливающий значения полей, проверяющий корректность данных, при ошибке генерирующий исключение;
 - метод show — метод вывода информации об устройстве в

формате "Device::show -> <name> <power> <load>".

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса Device

Функционал: параметризованный конструктор, устанавливающий значения полей, проверяющий корректность данных, при ошибке генерирующий исключение.

Параметры: строковый, name, имя устройства; целочисленный, power, мощность устройства; целочисленный, load, текущая нагрузка устройства.

Алгоритм конструктора представлен в таблице 1.

Таблица 1 – Алгоритм конструктора класса Device

№	Предикат	Действия	№ перехода
1	имя name пустое	сгенерировать исключение типа string со значением "Error: empty name"	Ø
			2
2	мощность power меньше или равна 0	сгенерировать исключение типа int со значением "-1"	Ø
			3
3	нагрузка load больше 100	сгенерировать исключение типа double со значением "100.0"	Ø
			4
4		присвоить переменной name текущего объекта значение входного параметра name	5
5		присвоить переменной power текущего объекта	6

№	Предикат	Действия	№ перехода
		значение входного параметра power	
6		присвоить переменной load текущего объекта значение входного параметра load	Ø

3.2 Алгоритм метода show класса Device

Функционал: метод вывода информации об устройстве в формате "Device::show -> <name> <power> <load>".

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода show класса Device

№	Предикат	Действия	№ перехода
1		вывод на консоль строки "Device::show -> ", значения переменной name, пробела, power, пробела, load и символа переноса строки	Ø

3.3 Алгоритм функции main

Функционал: организация работы программы.

Параметры: нет.

Возвращаемое значение: целочисленный - код возврата программы.

Алгоритм функции представлен в таблице 3.

Таблица 3 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		объявить целочисленную переменную n	2
2		считать значение из стандартного потока ввода в	3

№	Предикат	Действия	№ перехода
		переменную n	
3		пропустить чтение символа переноса строки после ввода n посредством вызова метода <code>cin.ignore()</code>	4
4		инициализировать целочисленную переменную i нулевым значением	5
5	значение индекса i меньше количества устройств n	объявить строковую переменную name	6
		вернуть 0	∅
6		объявить целочисленную переменную power	7
7		объявить целочисленную переменную load	8
8	строка начинается с пробела	присвоить переменной name пустую строку ""	9
		считать значение из стандартного потока ввода в переменную name	10
9		пропустить чтение пробела из стандартного потока ввода посредством вызова метода <code>cin.ignore()</code>	10
10		считать значение из стандартного потока ввода в переменные power и load	11
11		вызов метода <code>cin.ignore()</code> для очистки буфера перед следующей итерацией	12
12		создать объект device класса Device с аргументами name, power и load	13
13		вызов метода show объекта device	14
14	имя name пустое	вывод на консоль строки "Exception(string): " и сообщения об ошибке e	15
	мощность power меньше или равно 0	вывод на консоль строки "Exception(int): " и сообщения об ошибке e	15
	нагрузка load больше 100	вывод на консоль строки "Exception(double): " и сообщения об ошибке e	15

№	Предикат	Действия	№ перехода
		вывод на консоль строки "Exception: Unknown error"	15
15		инкрементировать i	5

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-5.

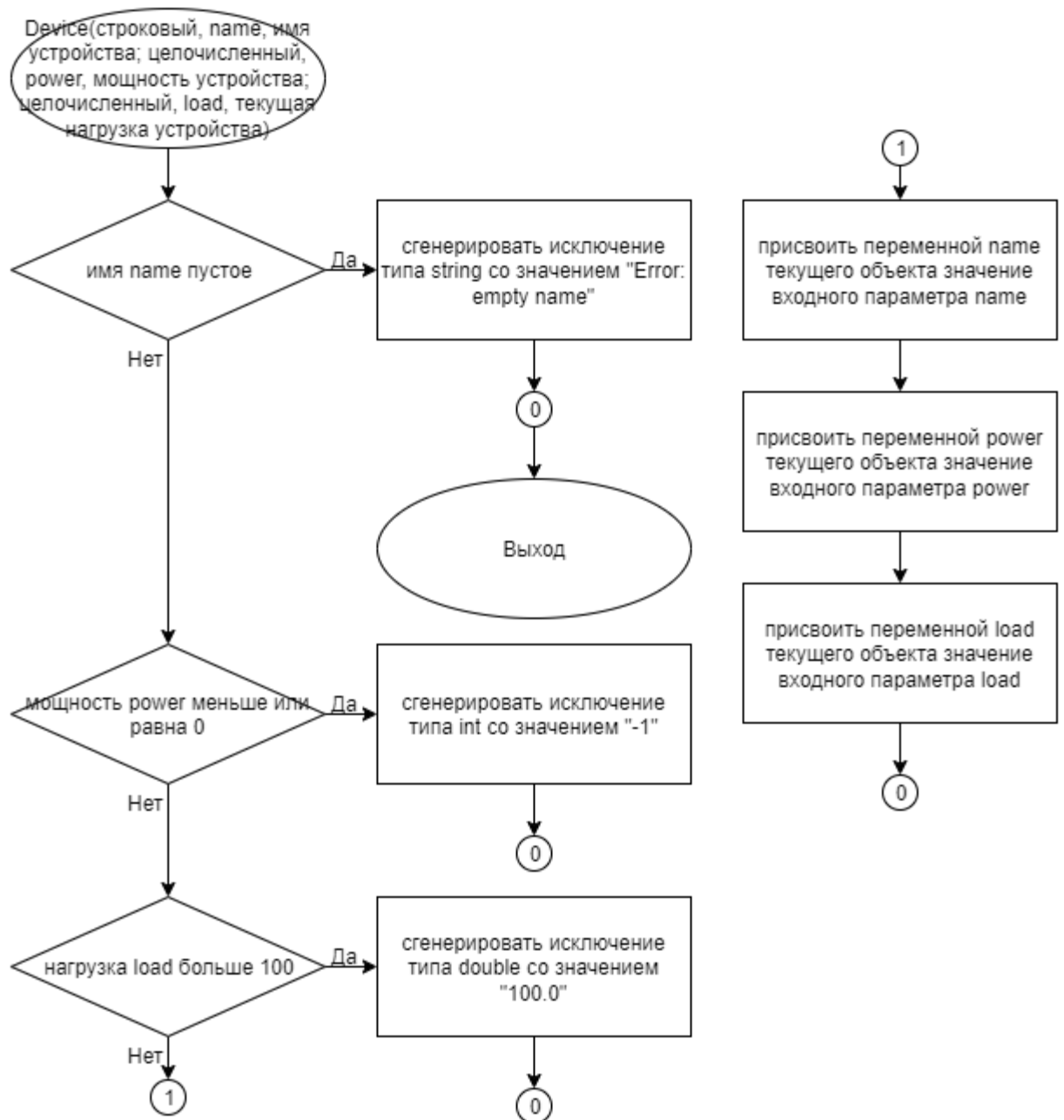


Рисунок 1 – Блок-схема алгоритма

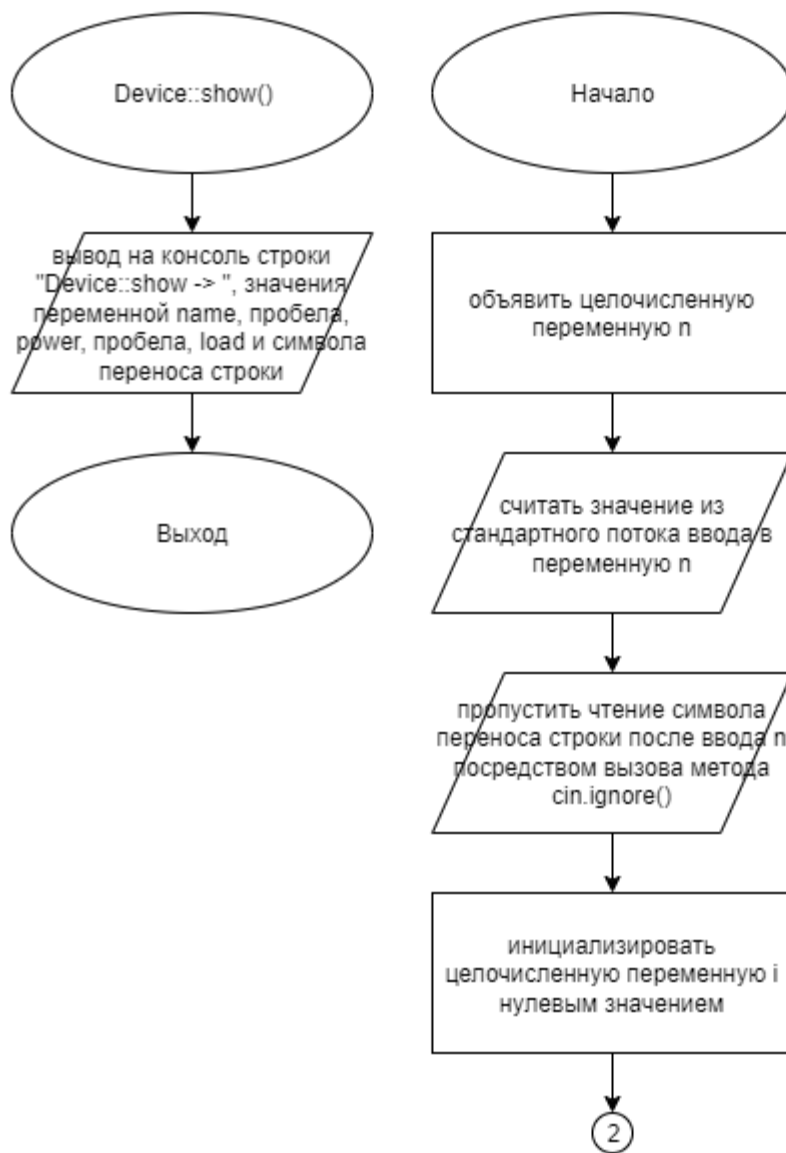


Рисунок 2 – Блок-схема алгоритма

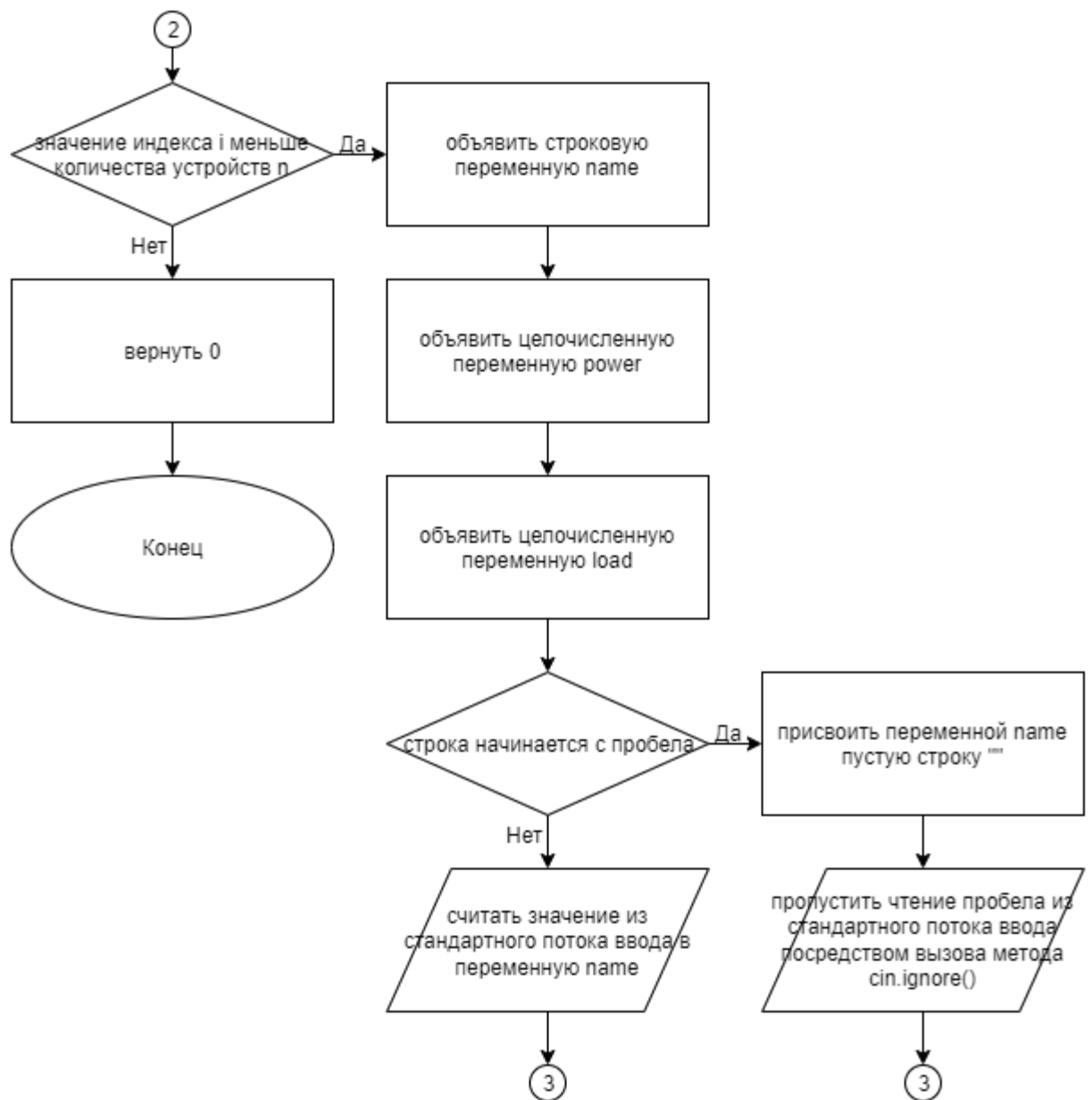


Рисунок 3 – Блок-схема алгоритма

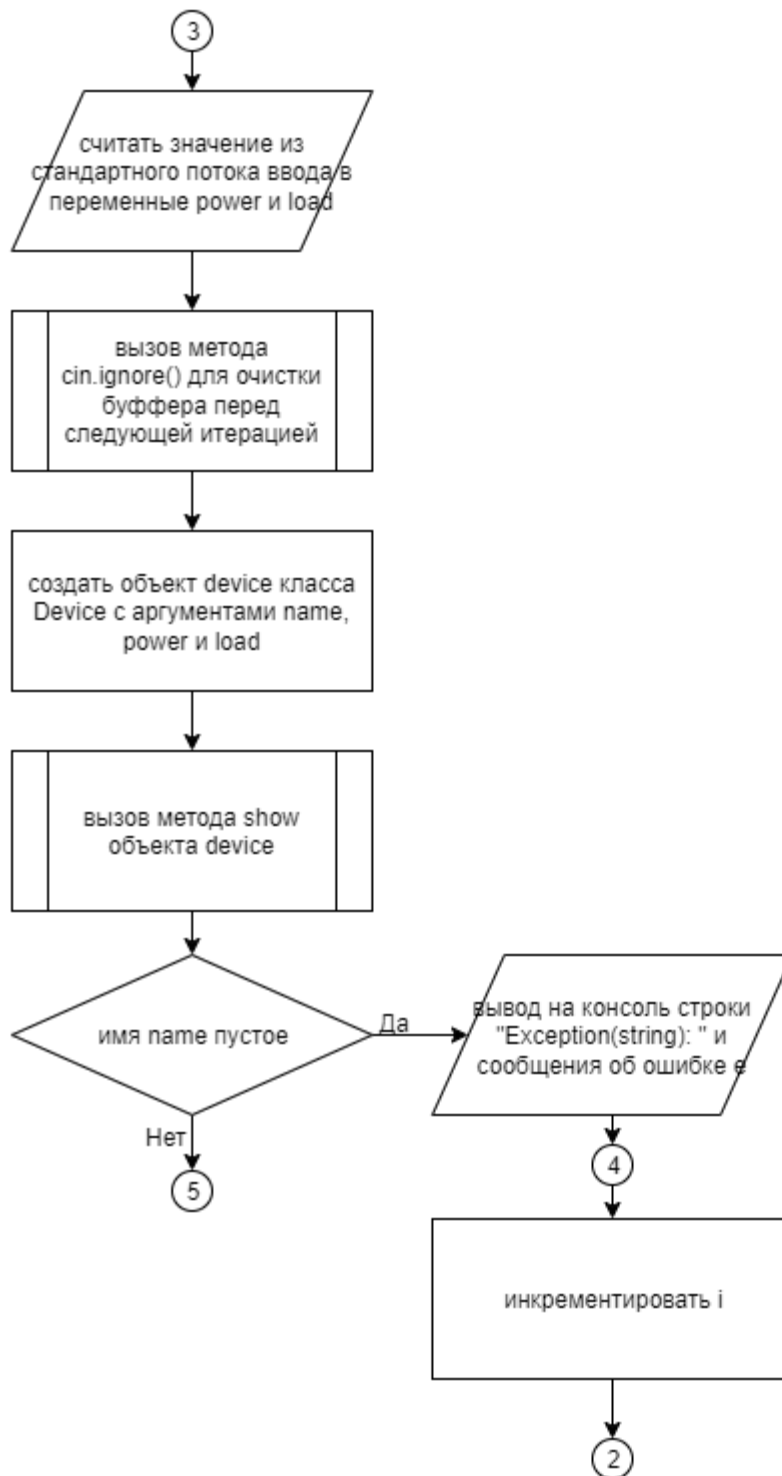


Рисунок 4 – Блок-схема алгоритма

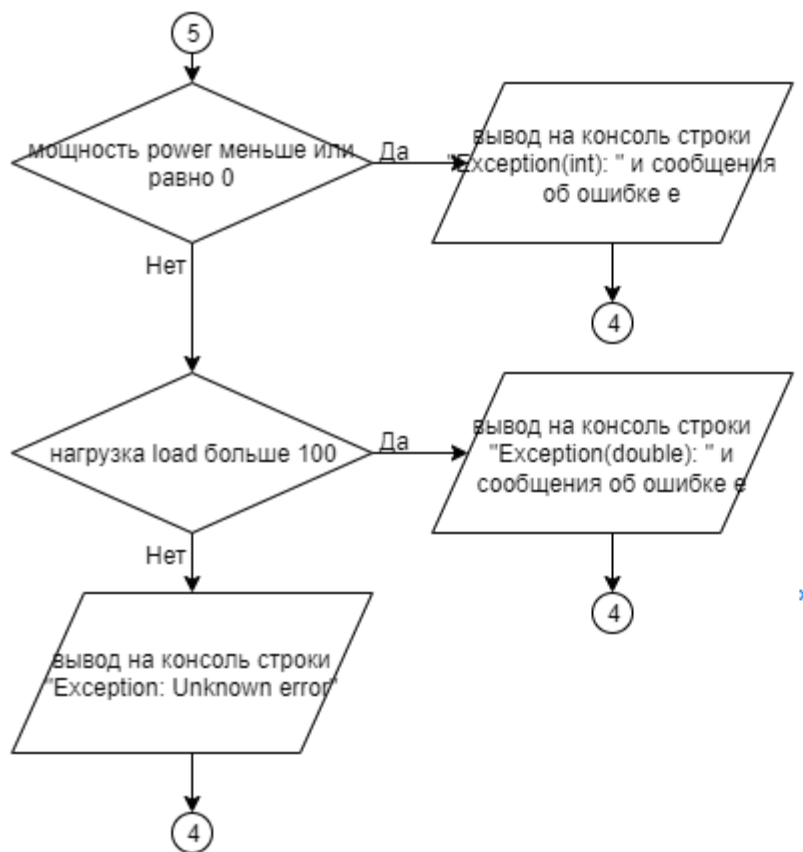


Рисунок 5 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Device.cpp

Листинг 1 – Device.cpp

```
#include "Device.h"
#include <iostream>
#include <string>
using namespace std;

Device::Device(string name, int power, int load) {
    if (name.empty()) throw string("Error: empty name");
    if (power <= 0) throw -1;
    if (load > 100) throw 100.0;

    this->name = name;
    this->power = power;
    this->load = load;
}

void Device::show() {
    cout << "Device::show -> " << name << " " << power << " " << load <<
endl;
}
```

5.2 Файл Device.h

Листинг 2 – Device.h

```
#ifndef __DEVICE__H
#define __DEVICE__H

#include <string>
using namespace std;

class Device {
private:
    string name;
    int power;
```

```

        int load;

public:
    Device(string name, int power, int load);
    void show();
};

#endif

```

5.3 Файл main.cpp

Листинг 3 – main.cpp

```

#include <stdlib.h>
#include <stdio.h>
#include <iostream>
#include <string>
#include "Device.h"
using namespace std;

int main()
{
    int n;
    cin >> n;
    cin.ignore();

    for (int i = 0; i < n; i++) {
        string name;
        int power;
        int load;

        if (cin.peek() == ' ') { // строка начинается с пробела
            name = "";          // имя пустое
            cin.ignore();        // пропуск пробела
        } else {
            cin >> name;         // ввод имени как обычно
        }

        cin >> power >> load;
        cin.ignore();

        try {
            Device device(name, power, load);
            device.show();
        } catch (string e) {
            cout << "Exception(string): " << e << endl;
        } catch (int e) {
            cout << "Exception(int): " << e << endl;
        } catch (double e) {

```

```
        cout << "Exception(double): " << e << endl;
    } catch (...) {
        cout << "Exception: Unknown error" << endl;
    }
}

return 0;
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 4.

Таблица 4 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
3 Router 50 30 0 20 Server -10 40	Device::show -> Router 50 30 Exception(string): Error: empty name Exception(int): -1	Device::show -> Router 50 30 Exception(string): Error: empty name Exception(int): -1
3 Router 3 20 Gamepad 1 10 Server 10 90	Device::show -> Router 3 20 Device::show -> Gamepad 1 10 Device::show -> Server 10 90	Device::show -> Router 3 20 Device::show -> Gamepad 1 10 Device::show -> Server 10 90

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoc_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).